

Setup queries for data load and all

1. Creating index for terminal\_id\_index

```
CREATE INDEX terminal_id_index
FOR (t:Terminal)
ON (t.id);
```

2. Creating. Customer\_id\_index

```
CREATE INDEX customer_id_index
FOR (c:Customer)
ON (c.id);
```

3. Next is importing data files, for that we need to copy all the files to instance folder and then the import folder.
4. Load customer data.

```
LOAD CSV WITH HEADERS FROM 'file:///customer_100MB.csv' AS row

CREATE (:Customer {
id: toInteger(row.CUSTOMER_ID),
x: toFloat(row.x_customer_id),
y: toFloat(row.y_customer_id),
mean_amount: toFloat(row.mean_amount),
```

```
std_amount: toFloat(row.std_amount),  
mean_tx_per_day: toFloat(row.mean_nb_tx_per_day),  
nb_terminals: toInteger(row.nb_terminals)  
});
```

## TERMINAL DATA

```
LOAD CSV WITH HEADERS FROM 'file:///terminal_100MB.csv' AS row  
  
CREATE (:Terminal {  
id: toInteger(row.TERMINAL_ID),  
x: toFloat(row.x_terminal_id),  
y: toFloat(row.y_terminal_id)  
});
```

## TRANSACTION DATA

```
LOAD CSV WITH HEADERS FROM 'file:///transactions_100MB.csv' AS row  
  
CALL (row) {  
MATCH (c:Customer {id: toInteger(row.CUSTOMER_ID)})  
MATCH (t:Terminal {id: toInteger(row.TERMINAL_ID)})  
  
CREATE (c)-[:MADE_TRANSACTION {  
tx_id: toInteger(row.TRANSACTION_ID),
```

```

amount: toFloat(row.TX_AMOUNT)
}}->(t)
} IN TRANSACTIONS OF 10000 ROWS;

```

5. THEN WE HAVE TO ADD FEW PROPERTIES IN TRANSACTION TO MAKE ALL WORK .

```

LOAD CSV WITH HEADERS FROM 'file:///transactions_100MB.csv' AS row

CALL (row) {
WITH row

MATCH ()-[r:MADE_TRANSACTION]->()
WHERE r.tx_id = toInteger(row.TRANSACTION_ID)

SET r.tx_datetime = row.TX_DATETIME,
r.tx_time_days = toInteger(row.TX_TIME_DAYS),
r.tx_time_seconds = toInteger(row.TX_TIME_SECONDS)
} IN TRANSACTIONS OF 10000 ROWS;

```

6. THEN WE FILL ALL THR VALUES IN TRANSACTIONS.

```

MATCH ()-[r:MADE_TRANSACTION]->()

```

```

WITH r

CALL {
  WITH r

  SET r.paymentMethod =

  CASE
  WHEN rand() < 0.33 THEN 'Credit Card'
  WHEN rand() < 0.66 THEN 'Mobile Payment'
  ELSE 'PayPal'
  END,

  r.promotionalOffer = rand() < 0.3,
  r.satisfactionRating = 1 + toInteger(rand()*5)
}

IN TRANSACTIONS OF 10000 ROWS;

```

8.

For outliers in query 5 we need outlier property too.

```
MATCH (c:Customer)-[r:MADE_TRANSACTION]->()
```

```
WITH c, r

CALL {

WITH c, r

SET r.isOutlier =

r.amount > (c.mean_amount + 3 * c.std_amount)

}

IN TRANSACTIONS OF 10000 ROWS;
```